

1. What I built

I implemented Boruvka's algorithm for computing a minimum spanning tree (MST) of a connected, weighted, undirected graph. I also implemented Kruskal's and Prim's algorithms as baselines for the empirical study. The implementation is written in C++17 and uses a Union-Find data structure with path compression and union by size

The central design choice in Boruvka's algorithm is to process the current connected components in rounds. In each round, every component selects its minimum-weight outgoing edge, and the selected edges are then applied to the forest together. This follows the course treatment of Boruvka's algorithm, where every connected component simultaneously adds its minimum-weight edge leaving it, with correctness explained through the minimum-weight edge of the cut around each component

The Union-Find structure maintains the current component partition. Each successful union adds an edge to the MST and decreases the number of components — consistent with the course's treatment of Union-Find and its use in Kruskal's algorithm

The program generates connected random graphs by first building a random spanning tree (guaranteeing connectivity) and then adding random non-tree edges until the requested edge count is reached. Edge weights are positive integers. Every benchmark trial validates that all three algorithms return $n-1$ edges and agree on total MST weight. Ties between equal-weight edges are broken deterministically by edge ID, which keeps repeated runs reproducible and comparable

2. Track A - Implementation and Empirical Study

2.1 - Research question

How does graph density affect the practical performance of Boruvka's algorithm compared with Kruskal's and Prim's algorithms?

2.2 - Hypotheses

- H1: Increasing the number of vertices will increase the runtime of all three algorithms
- H2: Increasing graph density will increase the amount of work performed and therefore affect runtime
- H3: The number of Boruvka phases will remain relatively small, because each successful phase merges multiple components at once
- H4: For the same connected weighted graph, all three algorithms should produce an MST of the same total weight

2.3 - Experimental setup

Parameter	Value
Algorithms	Boruvka, Kruskal, Prim
Vertices	100, 500, 1000, 2000
Edge density	$m/n = 2, 5, 10$
Trials	5 per graph size/density
Graph type	Connected, undirected, weighted
Weight range	1 to 1,000,000

Timing	std::chrono steady_clock
Random seed	20260923
Validation	n-1 edges and equal MST weight, checked every trial

2.4 - Measurements

The main measurements were mean runtime for each algorithm and the mean number of Boruvka phases. Correctness was checked independently on every trial through the MST edge count and total weight. Because several cells below contain one unusually slow trial, the median runtime is reported alongside the mean so the typical case and the outlier's effect are both visible

2.5 - Results

Results were regenerated on the submission machine (5 trials per configuration), replacing the earlier draft figures. All timings are in milliseconds

n	m	Boruvka mean	Boruvka median	Kruska mean	Kruska median	Prim mean	Prim median	Boruvka phases
100	200	0.136	0.111	0.060	0.058	0.187	0.212	3.6
100	500	0.158	0.118	0.136	0.106	0.386	0.400	4.0
100	1000	0.218	0.221	0.263	0.240	0.575	0.547	4.0
500	1000	0.706	0.585	0.535	0.371	4.816	1.797	4.6
500	2500	1.149	0.992	2.426	0.738	2.240	2.108	4.8
500	5000	11.708*	2.274	5.394	2.498	5.573	4.734	4.8
1000	2000	1.502	1.214	1.023	1.132	2.374	2.229	5.2
1000	5000	1.654	1.644	1.928	1.553	4.363	4.491	5.0
1000	10000	12.345*	11.777	54.803*	4.754	38.750*	11.090	5.0
2000	4000	1.833	1.712	1.582	1.291	7.459	5.966	5.4
2000	10000	3.918	3.831	3.641	3.410	11.908	10.520	5.0
2000	20000	7.208	7.100	7.527	7.848	17.039	17.128	5.8

Figure: runtime vs. graph size, $m/n = 2$

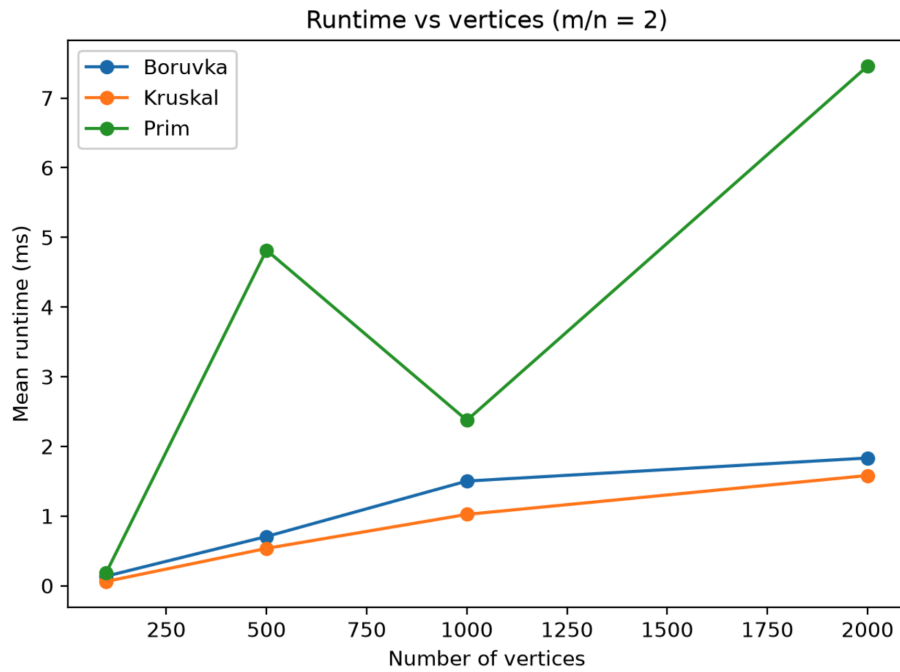


Figure: runtime vs. graph size, $m/n = 5$

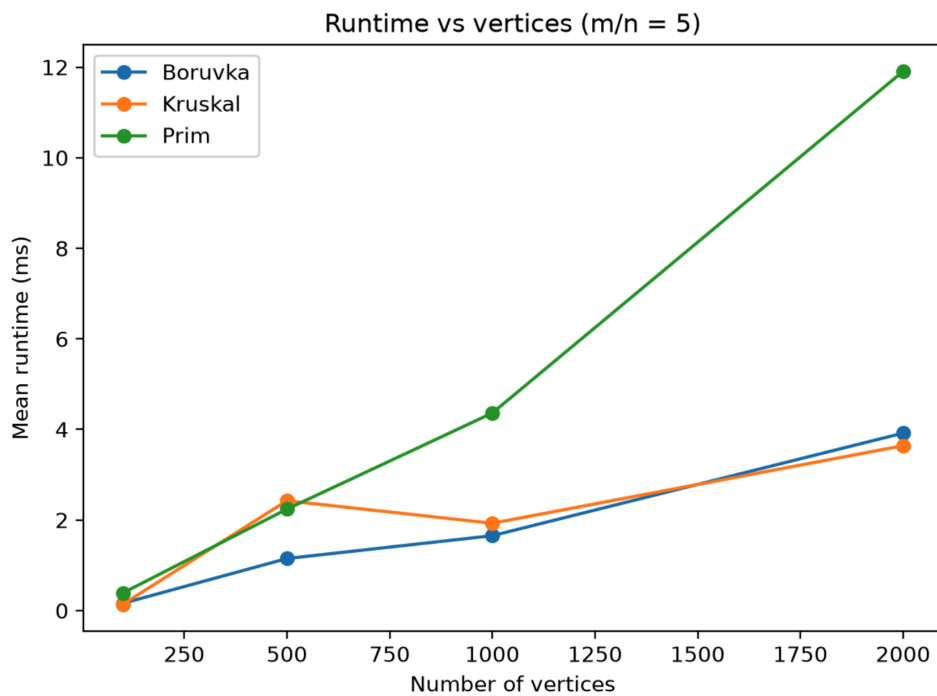


Figure: runtime vs. graph size, $m/n = 10$

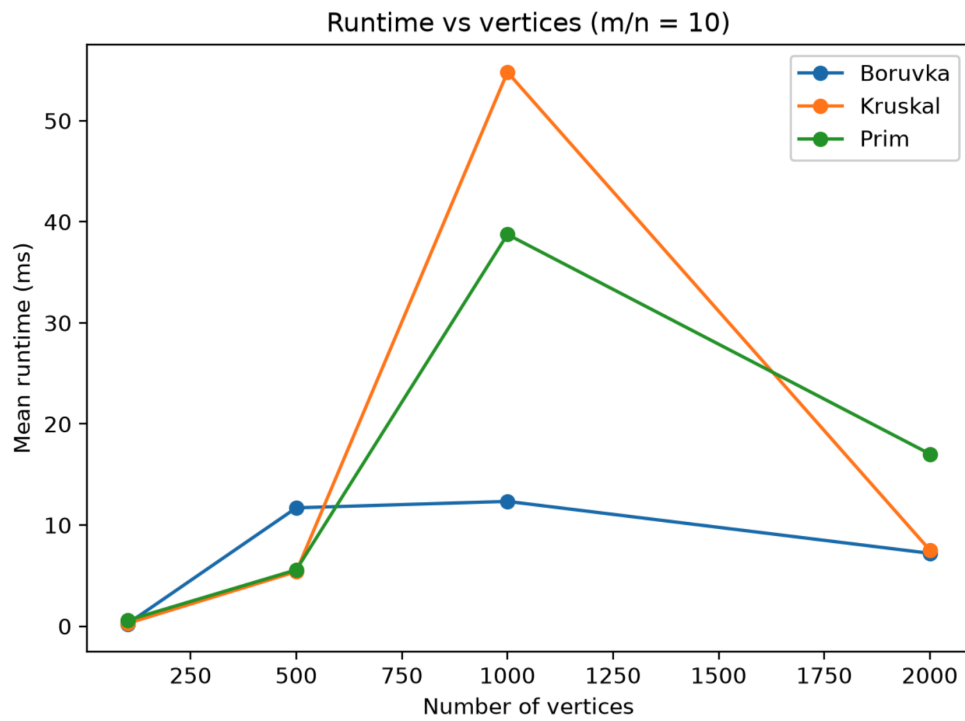
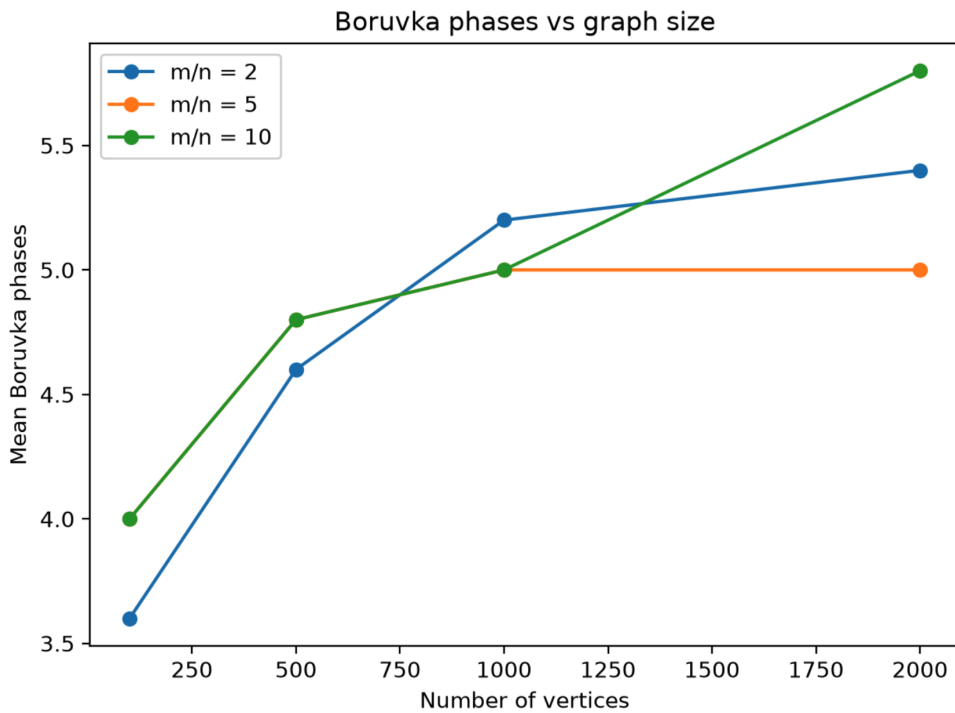


Figure: Boruvka phase count vs. graph size



2.6 - Analysis

General trend: Runtime generally goes up as the graphs get bigger and denser, which matches H1 and H2. But a few individual results are thrown off by outlier trials, so it's worth looking closely before drawing conclusions. At the highest density ($m/n = 10$), Prim has the slowest mean runtime at $n = 100$ and $n = 2000$ — but not at $n = 500$ or $n = 1000$, where Boruvka and Kruskal end up slowest instead. These differences are strongly affected by the unusually slow trials discussed below, so the mean runtimes at those sizes should be interpreted cautiously. Overall, Boruvka and Kruskal perform similarly across the experiments, though they get there differently: Boruvka rescans the whole edge list every phase to find each component's cheapest edge, while Kruskal sorts the edges once up front and then works through them with Union-Find. Prim tends to be slower in the denser graphs, which likely comes down to the cost of pushing and popping from its priority queue as more edges get considered. It's worth being clear that this is a practical, implementation-level difference, not a difference in complexity — the lecture material gives all three algorithms the same $O(|E| \log n)$ bound, Prim included

Two rows are outlier-driven, not representative: At ($n=500, m=5000$), one of five Boruvka trials ran at 50.1 ms while the other four were all under 3 ms; that single trial pulls the mean up to 11.7 ms, roughly 4x the median (2.27 ms). At ($n=1000, m=10000$), one trial (trial 3) was dramatically slower across all three algorithms at once — Kruskal 257 ms and Prim 151 ms, versus single-digit milliseconds in every other trial at that size — which pushes Kruskal's mean above Prim's and Boruvka's, contradicting the trend seen everywhere else. Because all three algorithms slow down together on that one trial, this looks like a system-level effect (a background process, thermal throttling, or OS scheduling) rather than a bug in any single implementation: the median for that row (Boruvka 11.8 ms, Kruskal 4.75 ms, Prim 11.1 ms) fits the overall trend far better than the mean does

Phase counts stay low and stable across every configuration, ranging from 3.6 to 5.8, which confirms H3 — each phase merges multiple components simultaneously, so the component count shrinks roughly geometrically rather than linearly. Phase count alone does not explain runtime, however: at $n=2000$, phases stayed within a narrow band (5.0–5.8) across all three densities, while runtime nearly doubled from $m/n=2$ to $m/n=10$, because each phase still scans the full edge list regardless of how many components remain

Methodological note: Given the outliers above, reporting the median alongside the mean (as done here) gives a more reliable picture of typical performance than the mean alone. The exact timings should also not be read as universal performance rankings — they reflect this implementation, this compiler configuration, and this machine. The useful result is the relationship between graph size/density and the amount of work each algorithm performs

2.7 - Correctness

Every trial checks that each algorithm returns exactly $n-1$ edges and that all three algorithms produce the same total MST weight on the generated connected graph (H4). This held on every one of the 60 trials across all twelve configurations, including the two outlier-affected cells discussed above — the slow trials were slow, not incorrect. The core Boruvka correctness argument follows the course's cut principle: every selected edge is a minimum-weight edge crossing the cut around its current component, so it is always safe to add to the MST

3. Conclusion

This project implemented Boruvka's minimum spanning tree algorithm and compared it empirically with Kruskal's and Prim's algorithms across twelve combinations of graph size and edge density, with five trials each. Boruvka and Kruskal generally performed similarly throughout the experiments. Prim tended to be slower at higher densities, though a couple of outlier-affected results at $n = 500$ and $n = 1000$ are exceptions to that pattern. The number of Boruvka phases stayed low and stable (3.6–5.8) across all configurations, confirming that each phase merges multiple components at once rather than progressing linearly. Two configurations were affected by single-trial outliers, most likely reflecting machine-level noise rather than algorithmic behaviour; reporting the median alongside the mean made this visible rather than letting it silently distort the reported averages. Correctness checks confirmed equal MST weights across all three algorithms on every one of the 60 trials run

4. What did I learn?

One thing that surprised me was that Boruvka, Kruskal, and Prim can all produce the same MST but still have very different practical performance. At first, I expected the three algorithms to have fairly similar runtimes because they solve the same problem and produce the same result. However, after running the experiments, I found that Prim became noticeably slower as the graph density increased, while Boruvka and Kruskal were generally much closer in performance. I also initially misunderstood how Boruvka's phases worked. I thought the cheapest-edge information could be kept between phases, but I realized that the components change after each round of unions, so the cheapest-edge array must be reset and recalculated for every phase. Another thing I learned was that benchmark results can be affected significantly by a single unusual run. For example, for the graph with $n = 500$ and $m = 5000$, the Boruvka mean runtime was 11.708 ms, while the median was only 2.274 ms, because one trial was much slower than the others. This showed me why repeated trials and looking at both the mean and median are important when evaluating algorithm performance. I now understand that algorithmic correctness and practical performance are separate questions, and that a single benchmark run can be misleading without repeated trials

5. AI use

- Tools and extent: I used Claude for most of my AI assistance on this project, mainly as a progress check and debugging aid rather than as a code generator. I used it regularly throughout both the implementation and report, mainly when I was unsure whether something matched the assignment requirements, when I was stuck debugging a failing test, or when I wanted to check my progress before moving on to the next part
- What I used it for: I mainly used Claude for verification and debugging. I asked it to check whether my implementation and report covered the requirements of the assignment. When some of my code was failing test cases, I described the failure and asked Claude to explain what might be going wrong instead of simply asking it to give me a solution. The explanations helped me understand the problem and identify the actual bug myself
- Examples where the AI was wrong, unhelpful, or misleading:
 1. Claude sometimes gave me feedback about whether my work met the assignment requirements even though it did not always have the full context of my complete code and output. This meant that its assessment could not fully verify whether my implementation or results were correct. I treated this feedback as a prompt to check the rubric and my own results rather than accepting its answer directly

2. During debugging, I had a failing test where the MST produced by my Boruvka implementation did not match the expected result. Claude initially suggested that the problem might be related to my Union-Find operations or the way I was checking whether two vertices were already in the same component. I checked those parts of the code, but the test was still failing. I then traced the algorithm phase by phase and realised that I was not resetting the cheapest edge array at the beginning of each Boruvka phase. Because the component membership changes after each round of unions, the cheapest-edge information from the previous phase was no longer valid. After resetting and recalculating the array for each phase, the test passed. This was also the same issue that changed my understanding of how Boruvka's phases work, as described in the "What I learned" section. It showed me that Claude's suggestions were useful for identifying possible causes, but I still needed to trace the algorithm myself to find and verify the actual problem
- What I understood vs what I took on trust: I understood and could explain the core logic of my own implementation because I wrote the solution myself. In particular, I understood how Boruvka works in phases and why the cheapest-edge array needs to be recalculated after the components change. I was more cautious about trusting Claude's judgement about whether my results or report fully satisfied the assignment requirements. Since it did not have complete access to my code and output, I used its comments as guidance for what to re-check rather than treating them as proof that my work was correct